

Interação com APIs

07

2026

SENAI - CRTI



Tratamento de Falhas e Mensagens

Construindo aplicações robustas e resilientes

APIs Falham no Mundo Real

Toda aplicação profissional enfrenta falhas de comunicação com servidores. O que diferencia um sistema amador de um sistema robusto não é a ausência de erros — é a **capacidade de responder a eles com elegância**.



Checkout Falhou

Pagamento não processado por timeout no servidor de pagamentos.



Login Expirou



Conexão Interrompida

Rede instável derrubou a requisição no meio do fluxo.



Pedido Não Carregou

Dashboard travado aguardando dados que nunca chegaram.



Mensagem-chave: Em software real, APIs falham — a aplicação precisa saber reagir.

O Problema da Equipe

O Cenário

Uma equipe desenvolveu um sistema Angular de cadastro e consulta de tarefas integrado a uma API REST. Durante os testes, surgiram problemas inesperados que comprometeram a experiência do usuário.

- Internet instável durante testes
- Timeout em requisições longas
- Endpoint temporariamente indisponível
- Usuário não autenticado acessando rotas protegidas
- Payload inválido enviado ao servidor
- Servidor interno falhando silenciosamente

A Pergunta-Problema

Como construir interfaces resilientes quando a API falha?

A aplicação precisa continuar **profissional, previsível e amigável** mesmo diante de erros. O usuário não pode ser deixado sem resposta ou com mensagens técnicas incompreensíveis.

- Esta situação representa um desafio real e recorrente no mercado de desenvolvimento de software.

Tipos de Falha em Aplicações Angular

Classificar o tipo de falha corretamente é o primeiro passo para tratá-la com precisão. Cada categoria exige uma estratégia de resposta diferente na interface.

Falhas de Cliente

- Dados inválidos no formulário
- Falha de validação local
- Rota ou endpoint incorreto

Falhas de Servidor

- Erro interno (500)
- Serviço indisponível (503)
- Timeout de processamento

Falhas de Rede

- Dispositivo offline
- Alta latência / DNS lento
- Conexão interrompida

Falhas de Segurança

- Token expirado (401)
- Acesso negado (403)

Status de Erro: Decifrando as Respostas do Servidor

O **status code HTTP** é o sinal que o servidor envia para orientar como a interface deve reagir. Conhecer esses códigos é essencial para implementar tratamentos precisos e mensagens relevantes ao usuário.

Erros do Cliente (4xx)

400	Bad Request — dados malformados
401	Unauthorized — não autenticado
403	Forbidden — sem permissão
404	Not Found — recurso inexistente
408	Timeout — servidor aguardou demais
422	Validation Error — dados inválidos
429	Too Many Requests — limite excedido

Erros do Servidor (5xx)

500	Internal Server Error
503	Service Unavailable

 **Regra de ouro:** O status code orienta a resposta da interface — cada código merece um tratamento específico e uma mensagem adequada ao usuário.

HttpErrorResponse: O Objeto de Falha do Angular

O Angular encapsula todas as falhas HTTP dentro do objeto `HttpErrorResponse`, importado de `@angular/common/http`. Esse objeto centraliza todas as informações necessárias para diagnóstico e tratamento da falha.

Importação

```
import { HttpErrorResponse } from  
'@angular/common/http';
```

Propriedades Disponíveis

status

Código numérico HTTP
(ex: 404, 500)

message

Descrição textual do erro

error

Corpo da resposta de erro
do servidor

headers

Cabeçalhos da resposta HTTP

url

URL que gerou o erro

Por que usar?

Com o `HttpErrorResponse`, o desenvolvedor tem acesso a todas as informações necessárias para:

- Exibir mensagens adequadas ao usuário
- Registrar logs detalhados para monitoramento
- Tomar decisões condicionais com base no status
- Redirecionar o usuário quando necessário (ex: 401 → login)

❏ Sempre tipar o parâmetro de erro como `HttpErrorResponse` para obter autocompletar e segurança de tipo no TypeScript.

Tratando Erros no subscribe

O método `subscribe()` aceita um objeto com três callbacks: `next` para dados recebidos, `error` para falhas e `complete` para o encerramento do Observable. O callback `error` é acionado automaticamente quando a requisição HTTP falha.

Código Completo

```
this.cepService.buscarCep(cep)
  .subscribe({
    next: (resposta) => {
      ...
    },
    error: () => {
      ...
    }
  });
```

Fluxo de Execução

- Requisição enviada
O Angular dispara a chamada HTTP ao endpoint configurado no service.
- Resposta recebida
Se bem-sucedida, o callback `next` é invocado com os dados.
- Falha detectada
Se houver erro HTTP, o callback `error` é invocado com o `HttpErrorResponse`.

⚠ Nunca deixe o callback `error` vazio — erros silenciosos são os mais difíceis de depurar em produção.

catchError: Centralizando o Tratamento de Falhas

O operador `catchError` do RxJS permite interceptar erros diretamente na cadeia de operadores do Observable, centralizando o tratamento no **service** e desacoplando essa responsabilidade do componente.

Importações Necessárias

```
import { catchError } from 'rxjs/operators';  
import { throwError } from 'rxjs';
```

Implementação no Service

```
return this.http.get(url).pipe(  
  catchError(error => {  
    return throwError(() => error);  
  })  
);
```

Por que centralizar no Service?

1

Reutilização

O tratamento vale para todos os componentes que consomem o mesmo service.

2

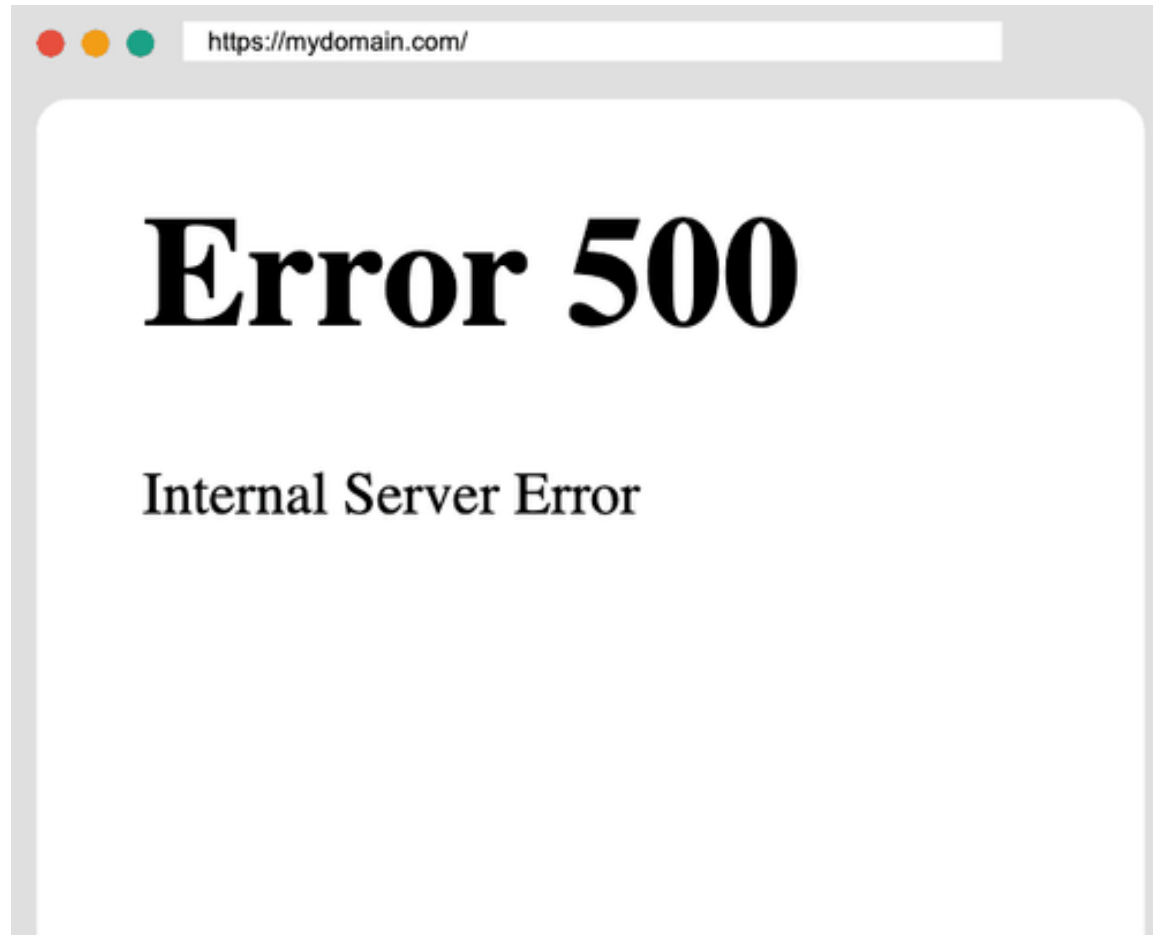
Separação de responsabilidades

O componente foca apenas na exibição; o service cuida da comunicação.

3

Manutenibilidade

Alterações no tratamento de erro não exigem mudanças em vários componentes.



UX DE ERRO

Feedback Visual ao Usuário

Um erro técnico **nunca deve ser traduzido diretamente** para o usuário. A interface tem o dever de transformar mensagens de sistema em comunicação humana, empática e orientada à ação.

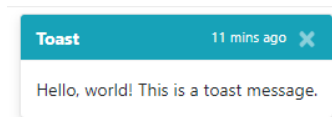
Boas Práticas de Feedback Visual

Elementos de Interface para Erros



Spinner / Loading

Indica processamento em andamento, evitando cliques duplicados.



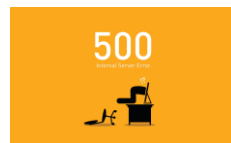
Toast / Snackbar

Notificação discreta e temporária para erros não críticos.



Botão Tentar Novamente

Devolve o controle ao usuário para reexecutar a ação.



Estado Vazio / Fallback

Tela amigável quando não há dados ou a requisição falhou.

Mensagens: Ruim vs. Bom

❌ **Exemplo ruim:** Internal Server Error

✅ **Exemplo bom:** Não foi possível carregar os dados.
Tente novamente.

A mensagem adequada:

- Explica o que aconteceu em linguagem simples
- Sugere uma ação ao usuário
- Não expõe detalhes técnicos sensíveis
- Mantém o tom profissional da aplicação

retry: Tentativas Automáticas de Requisição

O operador `retry` do RxJS reexecuta automaticamente uma requisição falha um número definido de vezes antes de propagar o erro. É especialmente útil em cenários de instabilidade de rede.

Importação e Uso

```
import { retry } from 'rxjs/operators';
```

```
return this.http.get(url).pipe(  
  retry(2)  
);
```

Neste exemplo, a requisição será executada até **3 vezes** no total: a original + 2 tentativas adicionais.

Quando Usar o retry?

✓ Use com retry

- Timeout de rede
- Rede instável ou intermitente
- Erros 503 temporários

✗ Não use com retry

- Erro de validação (422)
- Não autorizado (401/403)
- Dados malformados (400)



Retries em erros de cliente (4xx) são inúteis e podem sobrecarregar o servidor desnecessariamente.

Loading State: Os Quatro Estados de uma Tela

Interfaces profissionais mapeiam e gerenciam todos os estados possíveis de uma tela. Cada estado exige um comportamento visual distinto para guiar o usuário.

Loading

Dados sendo carregados. Exibir spinner ou skeleton screen. Desabilitar interações que dependam dos dados.

Error

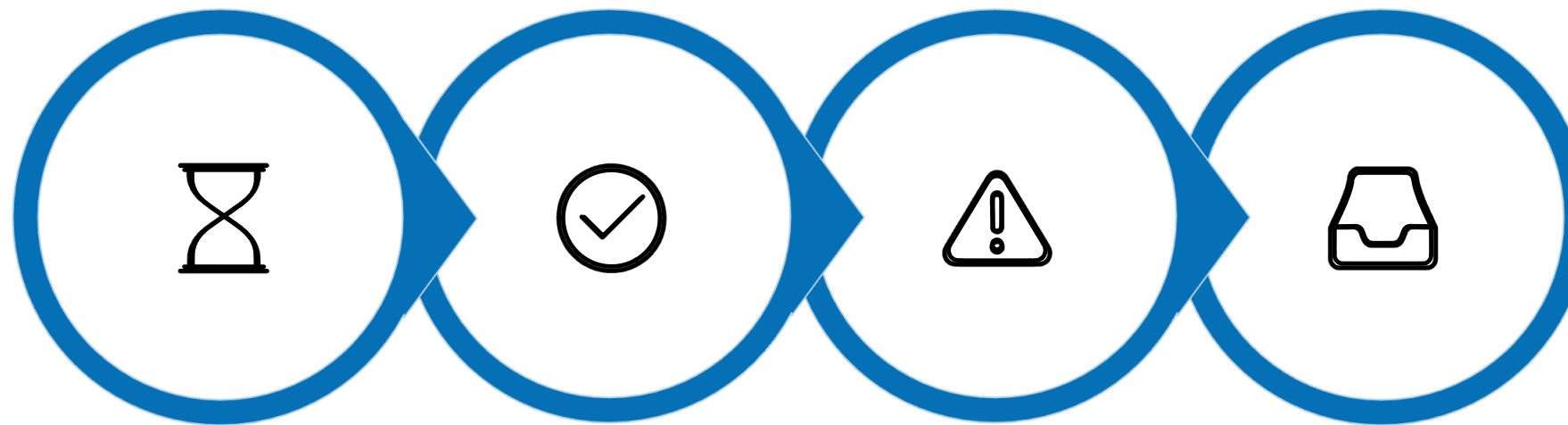
Falha na requisição. Exibir mensagem amigável, detalhes do problema e botão de retry para o usuário.

Success

Dados carregados com sucesso. Exibir o conteúdo normalmente. Estado padrão e esperado da aplicação.

Empty

Requisição bem-sucedida, mas sem dados. Exibir estado vazio com orientação de próxima ação ao usuário.



Loading

Success

Error

Empty

Caso Prático: Consulta de CEP com ViaCEP

A API pública **ViaCEP** é um excelente laboratório para praticar tratamento de falhas. Ela apresenta um comportamento interessante: retorna status `200 OK` mesmo para CEPs inválidos, mas com um corpo de resposta indicando erro.

Endpoint de Teste

```
https://viacep.com.br/ws/00000000/json/
```

Retorno da API para CEP Inválido

```
{ "erro": true }
```

⚠ **Atenção:** O status HTTP será `200`, mas o campo `erro: true` no corpo indica falha. Isso **não** dispara o callback `error` do `subscribe` automaticamente!

Como Tratar na UI?

- Verificar o corpo da resposta
No callback `next`, checar se `dados.erro === true`.
- Tratar como erro lógico
Mesmo com status 200, exibir mensagem de "CEP não encontrado".
- Exibir estado vazio
Não deixar a tela em branco — orientar o usuário a verificar o CEP digitado.



ATIVIDADE GUIADA

Mão na Massa: Consulta de CEP Robusta

Aplique na prática todos os conceitos da aula construindo uma aplicação Angular completa de consulta de CEP com tratamento profissional de falhas.

Checklist da Atividade

Implemente uma aplicação Angular de consulta de CEP utilizando a API pública ViaCEP, aplicando todos os recursos de tratamento de falha estudados nesta aula.

O que implementar

- ☐ Validação local do CEP
- ☐ Loading state
- ☐ Erro HTTP
- ☐ Erro lógico (erro: true)
- ☐ Mensagem amigável
- ☐ Renderização do endereço

Critérios de Avaliação

Funcionalidade

Todos os estados da tela funcionam corretamente?

Mensagens

As mensagens são amigáveis e orientadas à ação?

Código

O tratamento está centralizado adequadamente?

UX

A interface é previsível e profissional em todos os cenários?

Debate: Decisões Arquiteturais no Tratamento de Erros

Bons desenvolvedores não apenas codificam — eles refletem sobre as implicações das suas decisões. Vamos debater as questões mais importantes sobre estratégias de tratamento de falhas em aplicações Angular.

Log Técnico vs. Mensagem ao Usuário

Que informações devem ir para o log de monitoramento e quais para a tela do usuário? Como equilibrar transparência com segurança?

Retry Automático Sempre?

Em quais cenários o retry automático é uma boa ideia? Quando pode ser prejudicial ou até uma vulnerabilidade?

Onde Tratar o Erro?

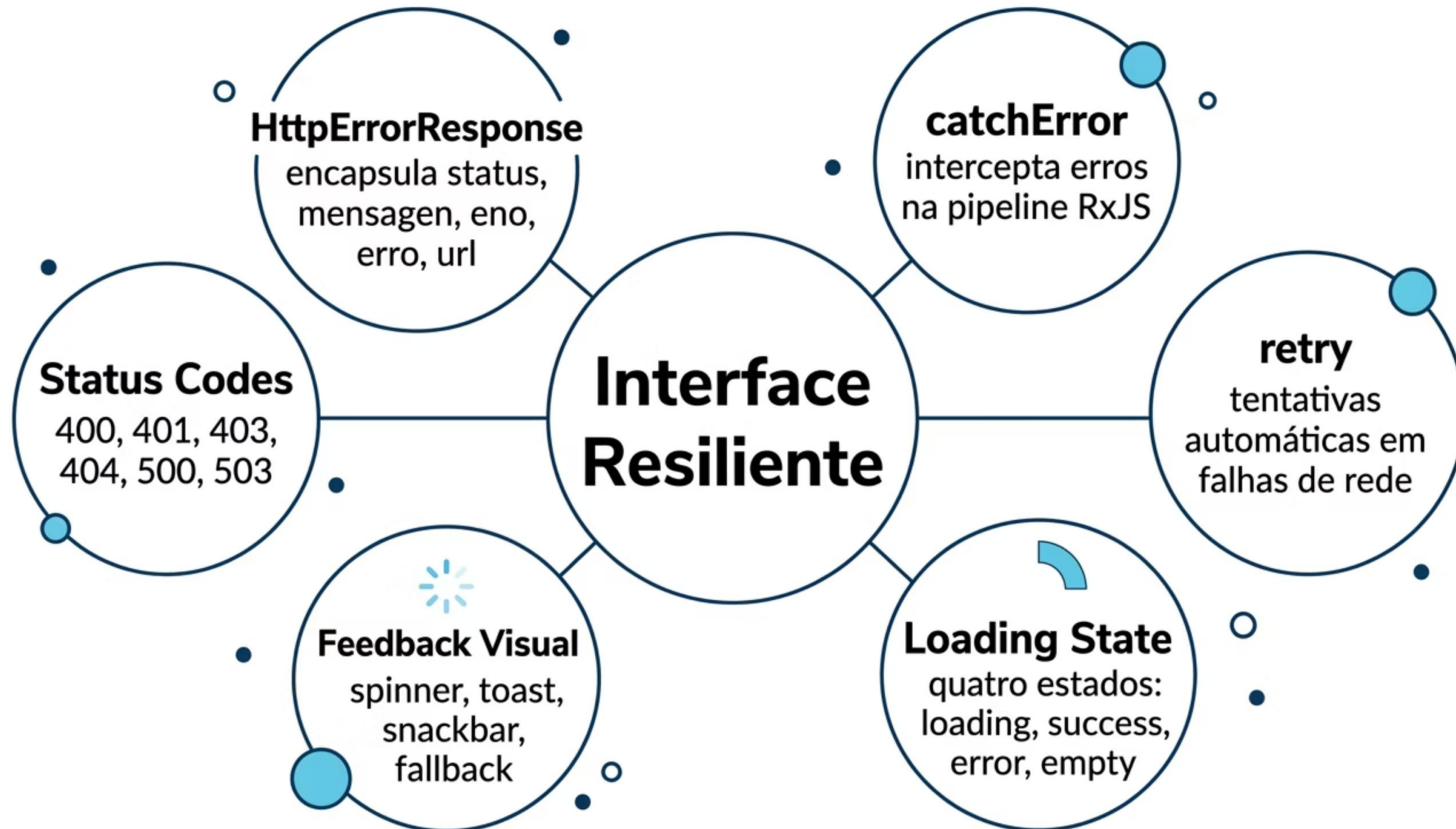
A responsabilidade pelo tratamento deve ficar no `service` ou no `component`? Quais são os prós e contras de cada abordagem?

Como Monitorar Falhas em Produção?

Quais ferramentas e estratégias permitem rastrear e analisar falhas de API em ambientes de produção com grande volume de usuários?

Mapa Conceitual da Aula

Todos os conceitos desta aula se conectam em torno de um objetivo central: construir aplicações Angular que respondam a falhas de forma inteligente, previsível e orientada ao usuário.



O que Você é Capaz de Fazer Agora

Ao concluir esta aula, você domina as ferramentas e práticas para construir aplicações Angular que não quebram diante de falhas — elas respondem com inteligência e profissionalismo.



Interpretar Falhas HTTP

Identificar e classificar erros por status code e tipo de origem.



Tratar subscribe error

Capturar falhas diretamente no fluxo do Observable com callbacks tipados.



Usar catchError

Centralizar o tratamento de falhas no service com operadores RxJS.



Aplicar retry

Configurar tentativas automáticas nos cenários corretos.



Implementar Loading State

Gerenciar os quatro estados da tela: loading, success, error e empty.



Criar Mensagens Amigáveis

Traduzir erros técnicos em comunicação clara e orientada ao usuário.

Resumo da Aula 7

Conceitos Aprendidos

01

Tipos de Falha

Cliente, servidor, rede e segurança.

02

Status Codes

4xx e 5xx — interpretação e resposta da UI.

03

HttpErrorResponse

Objeto Angular para encapsular falhas HTTP.

04

catchError + retry

Operadores RxJS para resiliência na pipeline.

05

Estados de Tela

Loading, Success, Error e Empty.

Princípio Central da Aula

Aplicações profissionais não evitam falhas — elas estão preparadas para elas. A resiliência é uma feature, não um detalhe.

